DRS - Dynamic Report System

DRS is an internal Python web application that delivers operational reports across your organization without requiring internet connectivity. It runs offline on a local server or personal computer within your corporate network, installed entirely from local files.

DRS is dynamic: every report is defined in web forms and stored directly in the DRS database. The database stores the SQL query, input parameters, grid columns, formatting rules, and permissions. Adding a new report or changing an existing one needs no backend code changes, no software rebuilds, and no server restarts. Users with design rights create, test, and publish reports on their own, while administrators only set up sources, groups, users, and grants once.

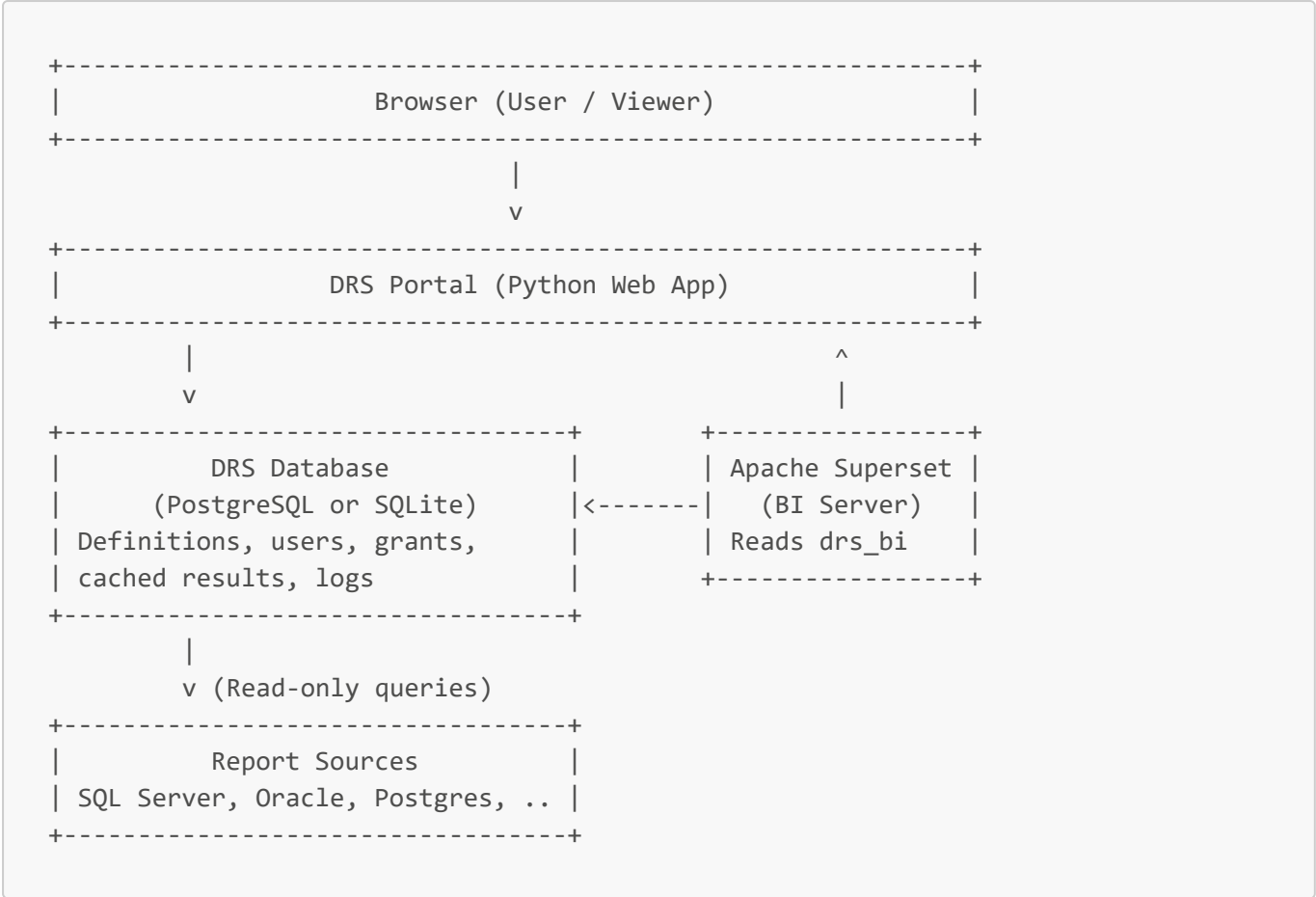
Links

What	Where
DRS portal (reports, Report design, administration)	`<DRS portal link>`
Apache Superset (dashboard design)	`<Superset link>`

Main features

- **Report sources** Connects to Microsoft SQL Server, Oracle, PostgreSQL, and SQLite using strictly read-only queries. Passwords for all datasources are stored encrypted at rest with secret keys.
- **Report views** Offers an interactive GRID view with sorting, filtering, totals, pagination, and exports to XLSX, CSV, and TXT. Supports custom HTML template layouts and Apache Superset BI dashboards embedded directly into report pages. Portal viewers need no Superset account.
- **Parameters** Supports text, numbers, dates, and booleans. Includes dynamic date defaults like the first day of the month (`@month_start`) or today (`@today`). Supports query-driven select lists and two-level dependent dropdown selects.
- **Result cache** Saves query results as snapshots in the DRS database with a configurable retention time. Requests within the retention period receive cached data immediately without querying the source database again.
- **Security and access control** Controls permissions through user accounts, roles, group grants, and report grants. Includes granular export and refresh rights, row filters based on user attributes, restricted reports, and comprehensive audit and access logs.
- **Self-service design** Includes a built-in Report design menu for drafting, testing, and publishing reports. Automatically generates BI datasets, manages group-based Superset access roles, attaches dashboards, and provides direct links into Superset.
- **Two languages** Supports switching between English and Vietnamese across all portal screens and forms.

System components



How a report runs

1. **Open report**: The user opens a report from the menu or navigates directly to its URL.
2. **Permission check**: DRS verifies that the user or their role holds an active grant on the report or its group.
3. **Parameters checked**: DRS validates submitted parameter values against expected data types, options lists, and required constraints.
4. **Cache check**: A cached result inside its retention time is served at once. Otherwise, the source is queried once within configured time and row limits.
5. **Snapshot saved**: The new query result is saved as a snapshot in the DRS database for caching and history.
6. **Row filter applied**: DRS filters result rows to match the user's assigned attributes.
7. **View rendered**: The user views the filtered result in the Grid, HTML layout, or embedded Dashboard tab.
8. **Exports generated**: Any export to XLSX, CSV, or TXT is generated directly from that same snapshot.
9. **Audit logging**: Every step, parameter set, query duration, row count, and export is recorded in the access log.

How a report is born

1. **Administrator sets up once**: The administrator registers the datasource, report group, and user grants once.
2. **Designer drafts SQL**: A report designer writes SQL in a web form, configures parameters and columns, and tests the query against the datasource.

3. **Designer publishes**: The designer publishes the report (or submits it for administrator approval if lacking publish rights).
4. **Immediate availability**: The report appears immediately in the menu of everyone with a grant on its group, with no backend code changes and no restarts.
5. **BI dataset creation**: Running the report for the first time automatically generates its BI dataset in the database.
6. **Dashboard authoring**: A dashboard designer builds charts and a dashboard in Apache Superset using that dataset.
7. **Dashboard attached**: The report designer attaches the dashboard on the BI page, and a Dashboard tab appears on the report page.

User roles

Role	What each does
Viewer	Signs in to the portal, selects parameters, views report tabs, and exports results.
Report designer	Writes SQL queries in the Report design menu, sets parameters and columns, tests queries, and publishes reports.
Dashboard designer	Builds charts and dashboards in Apache Superset using a dedicated Superset account.
Administrator	Sets up datasources, groups, users, and grants once; reviews draft submissions; and manages logs.

Technologies and open-source components

DRS itself is written in Python; every component below is open source and free of charge, used under its own licence.

Layer	Component	What it does in DRS	Licence
Language/runtime	Python	Programming language and application runtime	PSF-2.0
Web server and API	FastAPI	Web framework and API routing	MIT
Web server and API	Starlette	ASGI web application toolkit	BSD-3-Clause
Web server and API	Uvicorn	ASGI web server	BSD-3-Clause
Web server and API	Pydantic	Settings and data validation	MIT
Pages	Jinja2	HTML template rendering	BSD-3-Clause
Pages	Tabler UI kit	UI layout and styling	MIT
Pages	Tabler Icons	Interface icons	MIT

Layer	Component	What it does in DRS	Licence
Pages	Tabulator	Interactive grid for report results	MIT
Pages	htmx	Dynamic parameter and form updates	0BSD
Pages	Superset embedded SDK	Embedded dashboard viewer	Apache-2.0
Database access	SQLAlchemy	Database toolkit and query execution	MIT
Database access	Alembic	Database schema migrations	MIT
Database access	psycopg	PostgreSQL database driver	LGPL-3.0-only
Database access	pyodbc	ODBC driver for SQL Server	MIT
Database access	unixODBC (Linux)	ODBC driver manager used by pyodbc	LGPL-2.1
Database access	pymssql	Open-source SQL Server driver	LGPL-2.1
Database access	oracledb	Oracle Database driver	UPL-1.0 OR Apache-2.0
DRS database	PostgreSQL	Production database for DRS metadata, cache, and logs	PostgreSQL
DRS database	SQLite	Default local database and BI dataset storage	Public domain
Admin pages	sqladmin	Administration interface	BSD-3-Clause
Admin pages	WTForms	Form handling and validation in administration	BSD-3-Clause
Admin pages	Babel	Internationalization and localization	BSD-3-Clause
Security	argon2-cffi	Password hashing	MIT
Security	cryptography	Fernet encryption for datasource secrets	Apache-2.0 OR BSD-3-Clause
Security	itsdangerous	Cryptographic signing for session tokens	BSD-3-Clause
Security	Authlib	Single sign-on (SSO) authentication	BSD-3-Clause
Exports	XlsxWriter	Excel (XLSX) workbook generation	BSD-2-Clause
HTTP client	httpx	HTTP client for Superset API communication	BSD-3-Clause
BI	Apache Superset	Business intelligence dashboard server	Apache-2.0

Layer	Component	What it does in DRS	Licence
Containers	Docker Engine	Container runtime for DRS, PostgreSQL and Superset	Apache-2.0
Containers	Docker Compose	Multi-container deployment tool	Apache-2.0

Notes

- Microsoft ODBC Driver 18 for SQL Server (optional, only for the default SQL Server driver): free of charge under Microsoft's own licence terms, NOT open source; it is installed on your machine, not shipped with DRS. pymssql is an open-source alternative.
- Docker Engine on Linux is free (Apache-2.0).
- No component needs a paid licence or an internet connection at run time.

Where to go next

- DRS user guide - administrator
- DRS user guide - report designer